Bioinformatics

2025/2026

Day 2

1. Environment & Orchestration

Exercise 1

Use conda to create an environment named bioinfo_lab2 containing Python >=3.10, Emboss, and seqkit.

Exercise 2

Create a Makefile with the following targets: all, env, data, results, and clean. Consider the following file as a reference.

Makefile

```
None
# ==========================================
# Bioinformatics Lab 2: Makefile Template
# ==========================================
# Variables
PY = python3
CONDA_ENV = bioinfo_lab2
HUMAN_CDNA_URL =
https://ftp.ensembl.org/pub/release-115/fasta/homo_sapiens/cdna/Homo_sapiens
.GRCh38.cdna.all.fa.gz
HUMAN_CHRY_URL =
https://ftp.ensembl.org/pub/release-115/fasta/homo_sapiens/dna/Homo_sapiens.
GRCh38.dna.chromosome.Y.fa.gz
# TODO: Add the URLs
# release 115/version 54
DMEL_CDNA_URL =
DMEL_CHRY_URL =

# TODO: add the list of packages
```

```
PACKAGES =

# ==========================================================================
# Targets (dependencies are incomplete)
all: results/gc_comparison.tsv env init


init:
	mkdir -p data
	mkdir -p results

# ==========================================================================
# Exercise 1: Environment setup
env:
	conda create -c bioconda -n $(CONDA_ENV) $(PACKAGES) -y

# ==========================================================================
# Exercise 3 & 4: Data Acquisition
download_data: data/human_cdna.fa.gz data/dmel_cdna.fa.gz
data/dmel_chry.fa.gz data/human_chry.fa.gz

# Local files
HUMAN_CDNA=data/human_cdna.fa.gz
HUMAN_DNA=data/human_chry.fa.gz
DMEL_CDNA=data/dmel_cdna.fa.gz
DMEL_DNA=data/dmel_chry.fa.gz

$(HUMAN_CDNA):
	mkdir -p $(dir $@)
	curl -L $(HUMAN_CDNA_URL) -o $@

# COMPLETE
$(DMEL_CDNA):


# COMPLETE
$(HUMAN_DNA):


# COMPLETE
$(DMEL_DNA):


# ==========================================================================
# Exercise 5: Exploratory Analysis
# TODO: Add a target that outputs total base count and transcript count to a
text file
exploratory_stats: num_nucleotides.tsv num_seqs.csv num_Ns.csv
```

```
# Exercise 5.1
num_nucleotides.csv: $(HUMAN_CDNA).nnuc.csv $(DMEL_CDNA).nnuc.csv
	cat $^ > $@

# COMPLETE
%.fa.gz.nnuc.csv: %.fa.gz


# Exercise 5.2
num_seqs.csv: $(HUMAN_CDNA).nseqs.csv $(DMEL_CDNA).nseqs.csv
	cat $^ > $@

# COMPLETE
%.fa.gz.nseqs.csv: %.fa.gz


# Exercise 5.3
num_Ns.csv: $(HUMAN_CDNA).nN.csv $(DMEL_CDNA).nN.csv
	cat $^ > $@

# COMPLETE
%.fa.gz.nN.csv: %.fa.gz


#Exercise 5.4: Sequence Transformation
transform: $(HUMAN_CDNA).transformed $(DMEL_CDNA).transformed

# COMPLETE
%.fa.gz.transformed: %.fa.gz

# Exercise 5.5
# Rule to mask poly-A motifs
polyA_masked: $(HUMAN_CDNA:.fa.gz=.masked_polyA.fa.gz)
$(DMEL_CDNA:.fa.gz=.masked_polyA.fa.gz)

# COMPLETE
%.masked_polyA.fa.gz: %.fa.gz
	@echo "Masking Poly-A motifs in $<..."


# COMPLETE
#Exercise 5.6: 7096
results/GAATTC.num.txt: $(HUMAN_DNA)


# ==========================================================================
```

```
# Exercise 6.1 Python script

# Exercise 6.2
gc_stats: human_trans.gc.txt dmel_trans.gc.txt gc_stats_emboss

# COMPLETE
human_trans.gc.txt: $(HUMAN_CDNA)


dmel_trans.gc.txt: $(DMEL_CDNA)

gc_stats_emboss: human_trans.gc.emboss.txt dmel_trans.gc.emboss.txt

# COMPLETE
# assumes that the conda environment is active
human_trans.gc.emboss.txt: $(HUMAN_CDNA)


dmel_trans.gc.emboss.txt: $(DMEL_CDNA)



# TODO: Implement the rule to run gc_stats.py and capture time/memory
results/human.resources.tsv : data/human_chry.fa.gz gc_stats.py



# ======================================================================
# Exercise 7
TIME_MEM_FORMAT="%e\\t%M"

# COMPLETE
results/gc_bench_table.tsv: $(HUMAN_DNA) $(HUMAN_CDNA) $(DMEL_DNA)
$(DMEL_CDNA)
      echo "Tool   Input  Time (s)     Peak Memory (Kb)" > $@.tmp



# ======================================================================
# Exercise 7
# 7.1 python script seq_len.py

# 7.2 Determine the longest and shortest human transcript and its length.
# COMPLETE
results/human_cdna.lens.csv: $(HUMAN_CDNA) seq_len.py

# COMPLETE
ex7_2: results/human_cdna.lens.csv
```

```
	@echo -n "Longest human cdna - "

	@echo -n "Shortest human cdna - "


# 7.3 Determine the longest and shortest dmel transcript and its length.
# COMPLETE
results/dmel_cdna.lens.csv: $(DMEL_CDNA) seq_len.py


ex7_3: results/human_cdna.lens.csv
	@echo -n "Longest dmel cdna - "

	@echo -n "Shortest dmel cdna - "


# ======================================================================
# Exercise 8
# COMPLETE
results/human_chry.orfs.fa: $(HUMAN_DNA)


# COMPLETE
results/dmel_chry.orfs.fa: $(DMEL_DNA)


# stats
# COMPLETE
results/%.orfs.fa.stats: results/%.orfs.fa


orfs_stats: results/human_chry.orfs.fa.stats results/dmel_chry.orfs.fa.stats

# ====================================================================

clean:
	rm -rf data/* results/*
	@echo "Cleaned up project directory."
```

2. Data Acquisition (Ensembl)

Exercise 3

Complete your Makefile (make download_data) to obtain the latest cDNA FASTA files for Homo sapiens and Drosophila melanogaster (Ensembl release 115).

Exercise 4

Add rules to your Makefile to download the DNA sequences for:

- Human Chromosome Y (~7MB)
- D. melanogaster Chromosome Y (~900K)

Note: Ensure you select the "Top-level DNA" **unmasked** versions.

3. Fast Exploratory Analysis

Exercise 5

Use standard Linux command-line utilities (zcat, grep, sed, tr, wc, awk,...) to extract statistics from the compressed DNA and cDNA files obtained in Exercise 3.

5.1 Calculate the total number of nucleotides in each cDNA file.

5.2 Calculate the total number of sequences (transcripts) present in each file.

5.3 Calculate the total number of "N" characters (undetermined bases) across the entire transcriptome.

5.4 Create a new file with the suffix name ".transformed", for each cDNA file (fa.gz) from Exercise 3, that:

1) The sequence header does not contain spaces (everything after the first space is removed)
2) The sequence is converted to lowercase.
3) The new file should also be compressed with gzip to reduce disk space usage.

5.5 Soft Mask: Use sed or Python to convert all "Poly-A" motifs (length >= 5) to lowercase (e.g., AAAAA to aaaaa), for each cDNA file (**.fa.gz**) from Exercise 3. Create a gzip file with the suffix name **.masked_polyA.fa.gz**.

5.6 Quantify the frequency of the EcoRI restriction endonuclease recognition site (5' - GAATTC - 3') within the Human Chromosome Y DNA sequence.

- Target File: data/human_chry.fa.gz
- Challenge: The genomic sequence is "wrapped" (contains newlines). Your command must ensure that motifs split across two lines are not missed and that multiple motifs on a single line are counted individually.
- Expected Result: 7,096 occurrences.

4. Python Implementation

Exercise 6

6.1 GC Profiling. Write a **gc_stats.py** Python script to compute nucleotide composition for each sequence in the given fasta file. Statistics: i) Number of A, T, G and C and the GC percentage.

- **Algorithm:** $O(n)$ linear scan.
- **Input:** FASTA file (handle stdin via sys.stdin).
- **Output (TSV to Stdout):** Header \t #A \t #T \t #G \t #C \t GC%
- **Footer:** A final line starting with ---- followed by the **weighted mean** GC% ((nG + nC})/length(seq) * 100).

6.2 Summary Statistics. Use the script gc_stats.py on the cDNA files downloaded in exercise 3 and generate *human_trans.gc.txt* and *dmel_trans.gc.txt*.

6.2 Generate equivalent files using the EMBOSS tool that calculates GC content.

6.3 Compare the outputs. Does your "weighted mean" match the global average reported by EMBOSS? If not, why?

Exercise 7

Compare the computational efficiency (Time and Memory/RAM) of your custom Python implementation against the EMBOSS tool.

7.1 Use /usr/bin/time to capture the Elapsed Time and Maximum Resident Set Size (Peak Memory) for both your gc_stats.py script and the EMBOSS tool. You must process all four FASTA files retrieved in Exercises 3 and 4 (Human/Dmel cDNA and Human/Dmel Chromosomes).

> *Important*: Do not use the shell built-in time. You must use the binary located at /usr/bin/time. Check /usr/bin/time --help to understand the -f (format) string.

7.2 Automate the benchmarking using your Makefile. Use a format string to output the statistics directly into a TSV file with the format:

Tool Name\tInput Filename\tTime (s)\tPeak Memory (Kb)

- *Hint*: Use -f "%e\t%M" to capture the Command, Elapsed Time (seconds), and Peak Memory (Kilobytes).

7.3 Benchmarking Table. Fill in the following table with your results.

Tool	Input Filename	Time (s)	Peak Memory (Kb)	Input Size
gc_stats.py	**dmel_cdna.fa.gz**			89286901
gc_stats.py	**human_cdna.fa.gz**			742967561
gc_stats.py	**dmel_chry.fa.gz**			3667352
gc_stats.py	**human_chry.fa.gz**			57227415
EMBOSS Tool	**dmel_cdna.fa.gz**			89286901
EMBOSS Tool	**human_cdna.fa.gz**			742967561
EMBOSS Tool	**dmel_chry.fa.gz**			3667352
EMBOSS Tool	**human_chry.fa.gz**			57227415

7.4 Discussion

1. **Scaling:** How does the memory usage of your Python script change when moving from a smaller inputs to a larger inputs? Does it scale linearly with the file size or the length of the longest sequence?
2. **Implementation:** Explain why **/usr/bin/time** is necessary for memory tracking instead of the standard **time** command.

Exercise 8

Use the **getorf** tool to find potential genes and analyse the distribution of ORF lengths.

8.1 Use the EMBOSS tool getorf to find all ORFs in the Human Chromosome Y file with the following parameters:

- Minimum size: 300 nucleotides (-minsize 300).
- Find ORFs between START and STOP codons (-find 1).
- Output to a FASTA file.

conda run -n $(CONDA_ENV) getorf -sequence $<.fa -minsize 300 -find 1 -outseq $@.tmp && mv $@.tmp $@ #1: Translation of regions between START and STOP codons)

8.2 Use seqkit to analyse the output of getorf by generating some statistics based on the sequences in the file.

- Determine the total number of ORFs found and the length of the single longest ORF.
- Compare the number of ORFs found on Chromosome Y (largely heterochromatic/repetitive) versus the Human cDNA file.